

Laboratório de Concorrência · Aula 10

Heitor Henrique Scramim de Freitas · RA 24190382-2 · Turma A

Gerado em 09/10/2026, 20:21:21 · tempo registrado no cronômetro: 0:00

Preparação: Concorrência em cinco minutos

0.1 Com suas palavras: qual é a diferença entre sincronização de cooperação e de competição? Dê um exemplo do dia a dia (fora da computação) para cada uma.

Cooperação é quando uma tarefa precisa esperar a outra terminar alguma coisa pra poder continuar. Tipo num restaurante, o garçom só leva o prato quando o cozinheiro termina. Competição é quando as duas querem usar a mesma coisa e só dá pra uma usar por vez, tipo duas pessoas e um banheiro só em casa, uma tem que esperar a outra sair.

Prática 1: A condição de corrida

Previsão: Duas threads somam 1.000.000 cada. Que valor você espera que o programa imprima?

2.000.000, mas como não tem sincronização talvez dê menos.

1ª execução (n = 1.000.000)

1566815

2ª execução (n = 1.000.000)

1357685

3ª execução (n = 1.000.000)

1400611

Saída do programa

não respondido

The screenshot shows the OneCompiler IDE interface. The main editor displays a Java file named `Main.java` with the following code:

```
1 // Prática 1 - Condição de corrida (Sebesta 13.2.1, p. 547-549)
2 // Cole em https://onecompiler.com/java
3 public class Main {
4     static int contador = 0; // variável COMPARTILHADA
5
6     public static void main(String[] args) throws InterruptedException {
7         int n = 1_000_000; // repetições de cada thread
8         Runnable incrementa = () -> {
9             for (int i = 0; i < n; i++) contador++;
10        };
11        Thread t1 = new Thread(incrementa);
12        Thread t2 = new Thread(incrementa);
13        t1.start(); t2.start(); // as duas executam ao mesmo tempo
14        t1.join(); t2.join(); // espera as duas terminarem
15        System.out.println("esperado: " + 2 * n);
16        System.out.println("obtido: " + contador);
17    }
18 }
```

The right-hand pane shows the console output:

```
esperado: 2000000
obtido: 1566815
```

The top bar includes the OneCompiler logo, an 'Upgrade' button, and tabs for 'AI', 'Java', and 'Run'. The bottom status bar indicates 'Ready' and 'Terminal'.

The screenshot shows the OneCompiler web IDE interface. The main editor displays a Java file named `Main.java` with the following code:

```
1 // Prática 1 - Condição de corrida (Sebesta 13.2.1, p. 547-549)
2 // Cole em https://onecompiler.com/java
3 public class Main {
4     static int contador = 0; // variável COMPARTILHADA
5
6     public static void main(String[] args) throws InterruptedException {
7         int n = 1_000_000; // repetições de cada thread
8         Runnable incrementa = () -> {
9             for (int i = 0; i < n; i++) contador++;
10        };
11        Thread t1 = new Thread(incrementa);
12        Thread t2 = new Thread(incrementa);
13        t1.start(); t2.start(); // as duas executam ao mesmo tempo
14        t1.join(); t2.join(); // espera as duas terminarem
15        System.out.println("esperado: " + 2 * n);
16        System.out.println("obtido: " + contador);
17    }
18 }
```

The right-hand pane shows the console output:

```
esperado: 2000000
obtido: 1357685
```

The top bar includes the OneCompiler logo, an 'Upgrade' button, and tabs for 'AI', 'Java', and 'Run'. The bottom status bar shows 'Ready', 'Terminal', and system icons.

OneCompiler

Upgrade

AI

Java

Run

Save

Login

Main.java

1 // Prática 1 - Condição de corrida (Sebesta 13.2.1, p. 547-549)

2 // Cole em <https://onecompiler.com/java>

3 public class Main {

4 static int contador = 0; // variável COMPARTILHADA

5

6 public static void main(String[] args) throws InterruptedException {

7 int n = 1_000_000; // repetições de cada thread

8 Runnable incrementa = () -> {

9 for (int i = 0; i < n; i++) contador++;

10 };

11 Thread t1 = new Thread(incrementa);

12 Thread t2 = new Thread(incrementa);

13 t1.start(); t2.start(); // as duas executam ao mesmo tempo

14 t1.join(); t2.join(); // espera as duas terminarem

15 System.out.println("esperado: " + 2 * n);

16 System.out.println("obtido: " + contador);

17 }

18 }

Console

I/O

AI Agent

1.2s

esperado: 2000000

obtido: 1400611

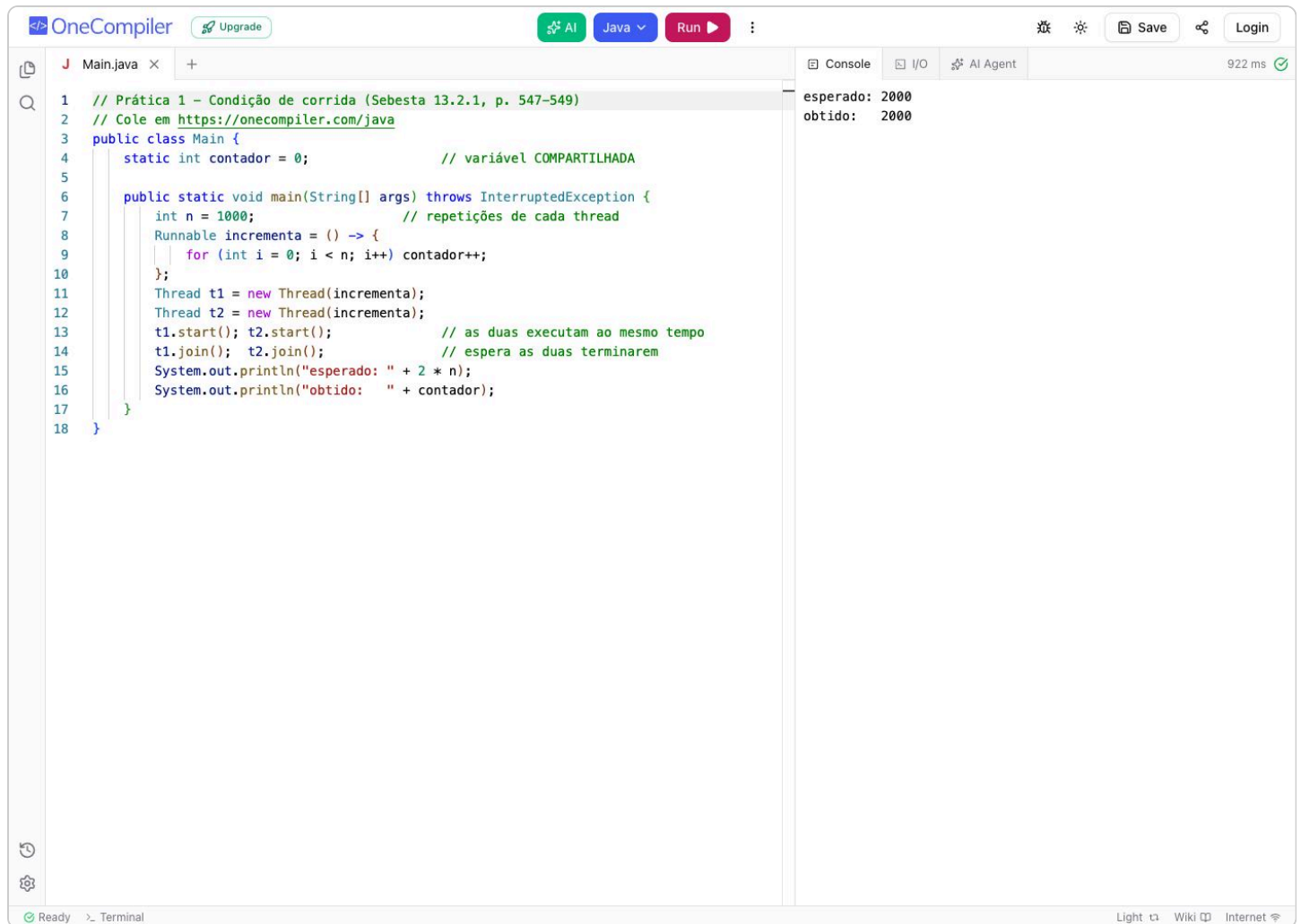
Ready

Terminal

Light

Wiki

Internet



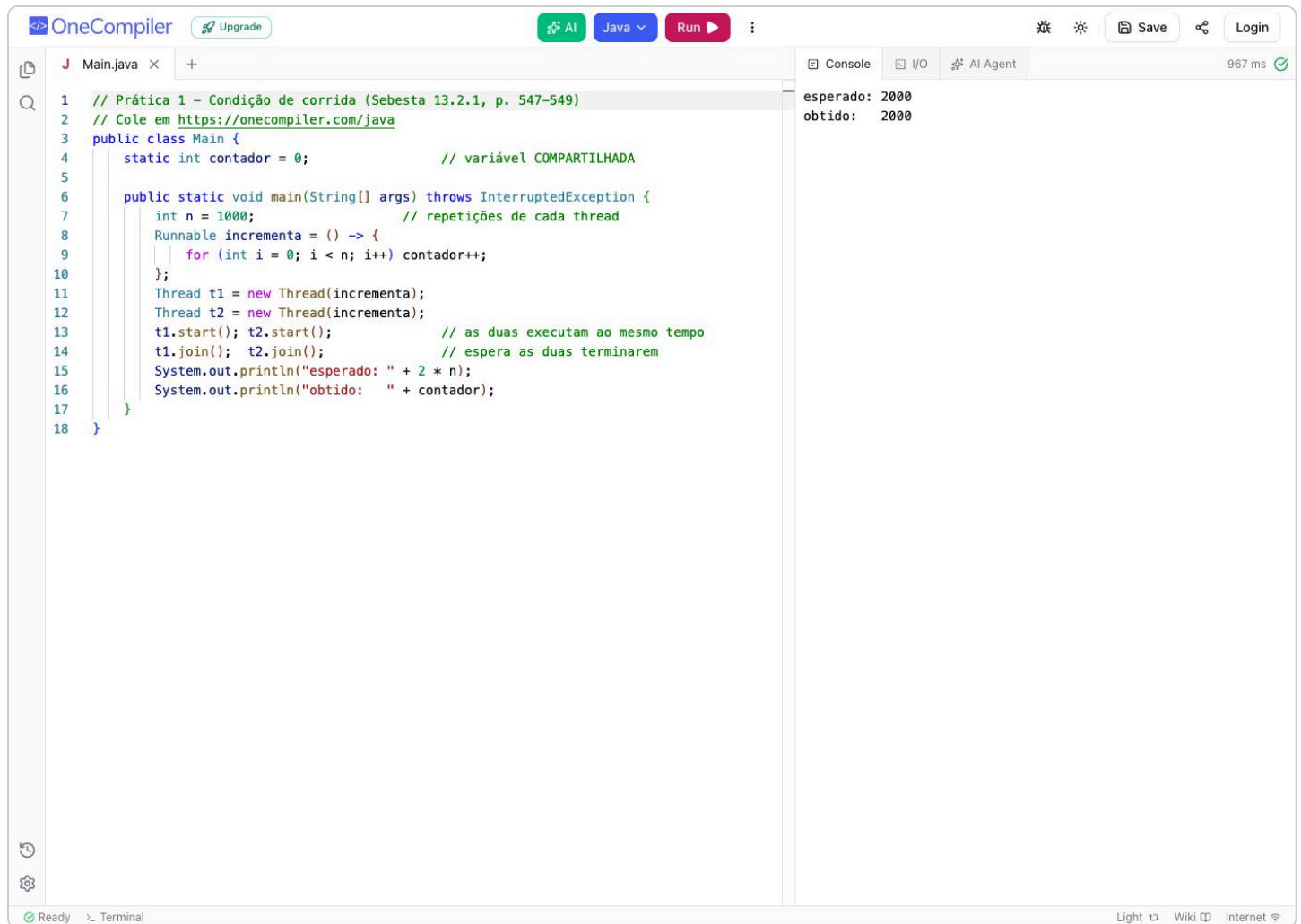
The screenshot shows the OneCompiler IDE interface. The main editor displays a Java file named `Main.java` with the following code:

```
1 // Prática 1 - Condição de corrida (Sebesta 13.2.1, p. 547-549)
2 // Cole em https://onecompiler.com/java
3 public class Main {
4     static int contador = 0; // variável COMPARTILHADA
5
6     public static void main(String[] args) throws InterruptedException {
7         int n = 1000; // repetições de cada thread
8         Runnable incrementa = () -> {
9             for (int i = 0; i < n; i++) contador++;
10        };
11        Thread t1 = new Thread(incrementa);
12        Thread t2 = new Thread(incrementa);
13        t1.start(); t2.start(); // as duas executam ao mesmo tempo
14        t1.join(); t2.join(); // espera as duas terminarem
15        System.out.println("esperado: " + 2 * n);
16        System.out.println("obtido: " + contador);
17    }
18 }
```

The right-hand pane shows the console output:

```
esperado: 2000
obtido: 2000
```

The bottom status bar indicates the application is "Ready" and shows a "Terminal" tab. The bottom right corner of the IDE shows "Light", "Wiki", and "Internet" icons.



```
1 // Prática 1 - Condição de corrida (Sebesta 13.2.1, p. 547-549)
2 // Cole em https://onecompiler.com/java
3 public class Main {
4     static int contador = 0; // variável COMPARTILHADA
5
6     public static void main(String[] args) throws InterruptedException {
7         int n = 1000; // repetições de cada thread
8         Runnable incrementa = () -> {
9             for (int i = 0; i < n; i++) contador++;
10        };
11        Thread t1 = new Thread(incrementa);
12        Thread t2 = new Thread(incrementa);
13        t1.start(); t2.start(); // as duas executam ao mesmo tempo
14        t1.join(); t2.join(); // espera as duas terminarem
15        System.out.println("esperado: " + 2 * n);
16        System.out.println("obtido: " + contador);
17    }
18 }
```

Console: esperado: 2000
obtido: 2000

1.1 Os valores bateram com o esperado? Foram iguais nas três execuções? Por que mudam de uma execução para outra?

Não bateu. Era pra dar 2.000.000 e deu entre 1,35 e 1,56 milhão, e cada vez saiu um número diferente. Isso acontece porque quem decide a hora que cada thread roda é o escalonador, e ele intercala as duas de um jeito diferente toda vez, então a quantidade de somas perdidas muda.

1.2 Explique, passo a passo, como uma atualização se perde. Use os três passos (ler, somar, gravar) e as duas threads.

O contador está em 10. A thread 1 lê 10. Antes dela gravar, a thread 2 também lê 10. A thread 1 soma e grava 11, e depois a thread 2 soma no valor que ela tinha lido (10) e grava 11 também. Ou seja, as duas somaram mas o contador só aumentou 1, a soma da thread 1 sumiu.

1.3 Com $n = 1000$, o que aconteceu? Por que isso torna esse tipo de erro perigoso em sistemas reais? Pense em como o programa se comportaria nos testes e em produção.

Com 1000 deu 2000 nas duas vezes. Acho que é porque a primeira thread termina tão rápido que nem dá tempo da segunda começar junto. O perigo é justamente esse: nos testes, com pouca coisa rodando, parece que está tudo certo, e o erro só aparece em produção quando tem muito acesso ao mesmo tempo, e aí é difícil de achar porque nem sempre acontece.

Prática 2: Todas as respostas possíveis (Figura 13.1)

Ordens testadas no simulador

AAABBB → 8

BBBAAA → 7

ABABAB → 6

AABBBA → 4

ABBBAA → 4

Saída do programa

não respondido

```

1 # Prática 2 - Todas as respostas possíveis da Figura 13.1 (Sebesta, p. 548)
2 # Cole em https://onecompiler.com/python
3 # TOTAL começa em 3. A tarefa A faz TOTAL + 1 e a tarefa B faz TOTAL * 2.
4 # Cada tarefa tem 3 passos: 1) ler TOTAL 2) calcular 3) gravar TOTAL.
5 from itertools import combinations
6
7
8 def executa(ordem):
9     total = 3
10    reg = {"A": 0, "B": 0}
11    passo = {"A": 0, "B": 0}
12    for t in ordem:
13        if passo[t] == 0:
14            reg[t] = total
15        elif passo[t] == 1:
16            reg[t] = reg[t] + 1 if t == "A" else reg[t] * 2
17        else:
18            total = reg[t]
19            passo[t] += 1
20    return total
21
22
23 print("AAABBB ->", executa("AAABBB")) # A inteira, depois B inteira
24 print("BBBAAA ->", executa("BBBAAA")) # B inteira, depois A inteira
25 print("ABABAB ->", executa("ABABAB")) # alternando passo a passo
26
27 resultados = {}
28 for posicoes in combinations(range(6), 3):
29     ordem = "".join("A" if i in posicoes else "B" for i in range(6))
30     resultados.setdefault(executa(ordem), []).append(ordem)
31
32 print("Ordens possíveis:", sum(len(v) for v in resultados.values()))
33 for valor in sorted(resultados):
34     ordens = resultados[valor]
35     print(f"TOTAL = {valor}: {len(ordens):2d} ordens, por exemplo {ordens[0]}")
  
```

AAABBB -> 8
 BBBAAA -> 7
 ABABAB -> 6
 Ordens possíveis: 20
 TOTAL = 4: 9 ordens, por exemplo AABBBBA
 TOTAL = 6: 9 ordens, por exemplo AABABBB
 TOTAL = 7: 1 ordens, por exemplo BBBBAAA
 TOTAL = 8: 1 ordens, por exemplo AAABBBB

2.1 Quantas ordens são possíveis e quantos resultados diferentes aparecem? Quantas ordens dão o resultado "correto" (8)?

Deu 20 ordens possíveis e 4 resultados diferentes (4, 6, 7 e 8). Só uma ordem dá 8, que é a AAABBB. 9 ordens dão 4, 9 dão 6 e 1 dá 7.

2.2 Escolha uma ordem que dá 4 e explique passo a passo por que o resultado é 4. O que aconteceu com a operação de B?

Na ordem AABBBA: A lê 3 e calcula 4, mas ainda não grava. Aí B lê 3 (porque A não gravou ainda), calcula 6 e grava 6. Por último A grava o 4 que ela tinha calculado, por cima do 6. A multiplicação do B se perdeu, porque o A sobrescreveu com um valor feito em cima do TOTAL antigo.

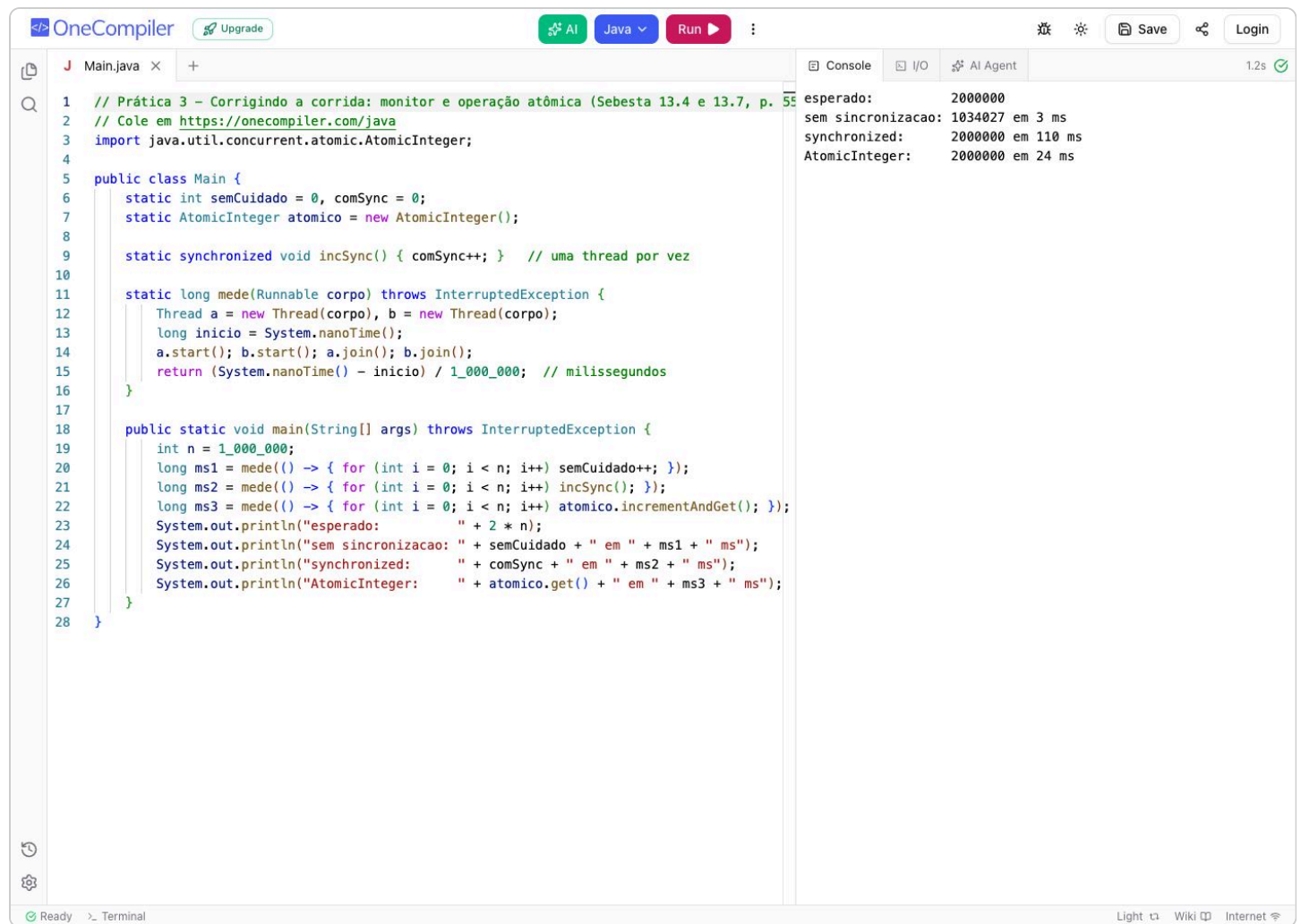
Prática 3: Corrigindo a corrida

Previsão: As três versões fazem a mesma conta. Qual você acha que será a mais rápida? E alguma será incorreta?

A sem sincronização deve ser a mais rápida mas vai dar errado, igual na prática 1. As outras duas devem dar certo.

Saída do programa

não respondido



The screenshot shows the OneCompiler IDE with a Java file named Main.java. The code implements a benchmark comparing three ways to increment a counter: using a semaphore, synchronization, and AtomicInteger. The main method runs each test 1,000,000 times and prints the results in milliseconds.

```
1 // Prática 3 - Corrigindo a corrida: monitor e operação atômica (Sebesta 13.4 e 13.7, p. 55)
2 // Cole em https://onecompiler.com/java
3 import java.util.concurrent.atomic.AtomicInteger;
4
5 public class Main {
6     static int semCuidado = 0, comSync = 0;
7     static AtomicInteger atomico = new AtomicInteger();
8
9     static synchronized void incSync() { comSync++; } // uma thread por vez
10
11     static long mede(Runnable corpo) throws InterruptedException {
12         Thread a = new Thread(corpo), b = new Thread(corpo);
13         long inicio = System.nanoTime();
14         a.start(); b.start(); a.join(); b.join();
15         return (System.nanoTime() - inicio) / 1_000_000; // milissegundos
16     }
17
18     public static void main(String[] args) throws InterruptedException {
19         int n = 1_000_000;
20         long ms1 = mede(() -> { for (int i = 0; i < n; i++) semCuidado++; });
21         long ms2 = mede(() -> { for (int i = 0; i < n; i++) incSync(); });
22         long ms3 = mede(() -> { for (int i = 0; i < n; i++) atomico.incrementAndGet(); });
23         System.out.println("esperado: " + 2 * n);
24         System.out.println("sem sincronizacao: " + semCuidado + " em " + ms1 + " ms");
25         System.out.println("synchronized: " + comSync + " em " + ms2 + " ms");
26         System.out.println("AtomicInteger: " + atomico.get() + " em " + ms3 + " ms");
27     }
28 }
```

The output in the console shows the following results:

```
esperado: 2000000
sem sincronizacao: 1034027 em 3 ms
synchronized: 2000000 em 110 ms
AtomicInteger: 2000000 em 24 ms
```

The screenshot shows the OneCompiler IDE with a Java file named Main.java. The code implements a concurrent program with two threads, 'a' and 'b', that perform a series of operations on shared variables. The output in the console shows the results of these operations, comparing the 'sem sincronizacao' (no synchronization) version with the 'synchronized' and 'AtomicInteger' versions.

```

1 // Prática 3 – Corrigindo a corrida: monitor e operação atômica (Sebesta 13.4 e 13.7, p. 55)
2 // Cole em https://onecompiler.com/java
3 import java.util.concurrent.atomic.AtomicInteger;
4
5 public class Main {
6     static int semCuidado = 0, comSync = 0;
7     static AtomicInteger atomico = new AtomicInteger();
8
9     static synchronized void incSync() { comSync++; } // uma thread por vez
10
11     static long mede(Runnable corpo) throws InterruptedException {
12         Thread a = new Thread(corpo), b = new Thread(corpo);
13         long inicio = System.nanoTime();
14         a.start(); b.start(); a.join(); b.join();
15         return (System.nanoTime() - inicio) / 1_000_000; // milissegundos
16     }
17
18     public static void main(String[] args) throws InterruptedException {
19         int n = 1_000_000;
20         long ms1 = mede(() -> { for (int i = 0; i < n; i++) semCuidado++; });
21         long ms2 = mede(() -> { for (int i = 0; i < n; i++) incSync(); });
22         long ms3 = mede(() -> { for (int i = 0; i < n; i++) atomico.incrementAndGet(); });
23         System.out.println("esperado:          " + 2 * n);
24         System.out.println("sem sincronizacao: " + semCuidado + " em " + ms1 + " ms");
25         System.out.println("synchronized:    " + comSync + " em " + ms2 + " ms");
26         System.out.println("AtomicInteger:   " + atomico.get() + " em " + ms3 + " ms");
27     }
28 }

```

Console Output:

```

esperado:          2000000
sem sincronizacao: 1005468 em 2 ms
synchronized:     2000000 em 156 ms
AtomicInteger:    2000000 em 26 ms

```

3.1 Quais versões deram o valor correto? Compare os tempos: por que a versão correta costuma ser mais lenta que a versão sem sincronização?

O synchronized e o AtomicInteger deram 2.000.000 certinho, a sem sincronização deu uns 1 milhão. No tempo a sem sincronização levou 2-3 ms, o Atomic uns 25 ms e o synchronized 110-156 ms. Fica mais lento porque com a trava uma thread tem que esperar a outra sair da seção crítica, então elas não rodam de fato em paralelo, e pegar e soltar a trava a cada incremento também tem custo. O Atomic é mais rápido que o synchronized porque usa uma instrução atômica do processador em vez de travar a thread.

3.2 Uma transferência bancária tira dinheiro de uma conta e coloca em outra. Um AtomicInteger em cada saldo resolve? Por quê? O que seria preciso?

Não resolve, porque a transferência são duas operações (tirar de uma e colocar na outra) e cada uma ser atômica não deixa as duas juntas atômicas. No meio delas outra thread pode ler os saldos errados ou mexer na mesma conta. Teria que travar as duas contas durante a transferência toda (sempre na mesma ordem pra não dar impasse) ou usar uma transação no banco.

Prática 4: Impasse (deadlock)

Previsão: Leia o código: A pega X e depois Y; B pega Y e depois X. As mensagens "A terminou" e "B terminou" vão aparecer?

Acho que não, as duas vão ficar travadas esperando uma pela outra.

Saídas (antes e depois da correção)

não respondido

The screenshot shows the OneCompiler IDE with a Java file named `Main.java`. The code implements a deadlock scenario between two threads, A and B. Thread A holds lock X and is waiting for lock Y. Thread B holds lock Y and is waiting for lock X. Both threads call a `pausa()` method that sleeps for 100ms. The `main` method starts both threads, sleeps for 1000ms, and then checks for deadlocked threads using `ManagementFactory.getThreadMXBean().findDeadlockedThreads()`. The output in the console is `IMPASSE: 2 threads presas`.

```
1 // Prática 4 - Impasse (deadlock) (Sebesta 13.2.1, p. 551)
2 // Cole em https://onecompiler.com/java
3 import java.lang.management.ManagementFactory;
4
5 public class Main {
6     static final Object X = new Object(), Y = new Object(); // dois recursos
7
8     static void pausa() {
9         try { Thread.sleep(100); } catch (InterruptedException e) { }
10    }
11
12    public static void main(String[] args) throws Exception {
13        Thread a = new Thread(() -> {
14            synchronized (X) { pausa(); synchronized (Y) { System.out.println("A terminou")
15            };
16        });
17        Thread b = new Thread(() -> {
18            synchronized (Y) { pausa(); synchronized (X) { System.out.println("B terminou")
19            };
20        });
21        a.start(); b.start();
22        Thread.sleep(1000); // espera 1 segundo
23        long[] presas = ManagementFactory.getThreadMXBean().findDeadlockedThreads();
24        System.out.println(presas == null ? "sem impasse" : "IMPASSE: " + presas.length + " threads presas");
25        System.exit(0); // encerra o programa
26    }
```

Console output: IMPASSE: 2 threads presas

```

1 // Prática 4 – Impasse (deadlock) (Sebesta 13.2.1, p. 551)
2 // Cole em https://onecompiler.com/java
3 import java.lang.management.ManagementFactory;
4
5 public class Main {
6     static final Object X = new Object(), Y = new Object(); // dois recursos
7
8     static void pausa() {
9         try { Thread.sleep(100); } catch (InterruptedException e) { }
10    }
11
12    public static void main(String[] args) throws Exception {
13        Thread a = new Thread(() -> {
14            synchronized (X) { pausa(); synchronized (Y) { System.out.println("A terminou"); }
15        });
16        Thread b = new Thread(() -> {
17            synchronized (X) { pausa(); synchronized (Y) { System.out.println("B terminou"); }
18        });
19        a.start(); b.start();
20        Thread.sleep(1000); // espera 1 segundo
21        long[] presas = ManagementFactory.getThreadMXBean().findDeadlockedThreads();
22        System.out.println(presas == null ? "sem impasse" : "IMPASSE: " + presas.length + " threads presas");
23        System.exit(0); // encerra o programa
24    }
25 }

```

Console output:

```

A terminou
B terminou
sem impasse

```

4.1 Por que as duas threads ficaram presas? Descreva o que cada uma tinha e o que cada uma esperava.

A pegou o X e ficou esperando o Y, e nesse meio tempo B pegou o Y e ficou esperando o X. Como cada uma só solta o que tem quando terminar, e nenhuma consegue terminar, ficam as duas paradas pra sempre.

4.2 Por que pegar os recursos sempre na mesma ordem elimina o impasse? Dê um exemplo de sistema real em que isso poderia acontecer.

Se as duas pegam primeiro o X, quem conseguir o X segue e pega o Y também, e a outra fica esperando sem estar segurando nada. Quando a primeira termina, a segunda consegue. Um exemplo real seria num banco, uma transferência de João pra Maria e outra de Maria pra João ao mesmo tempo, cada uma travando primeiro a conta de quem está enviando. Dá pra resolver travando sempre pela ordem do número da conta, por exemplo.

Prática 5: Mensagens em vez de memória compartilhada

Previsão: São 6 tarefas de 100 ms divididas entre 3 trabalhadores. Quanto tempo você espera que leve no total? E com 1 trabalhador? E com 6?

Com 3 uns 200 ms, com 1 uns 600 ms e com 6 uns 100 ms.

Tempo com 1 trabalhador

600ms

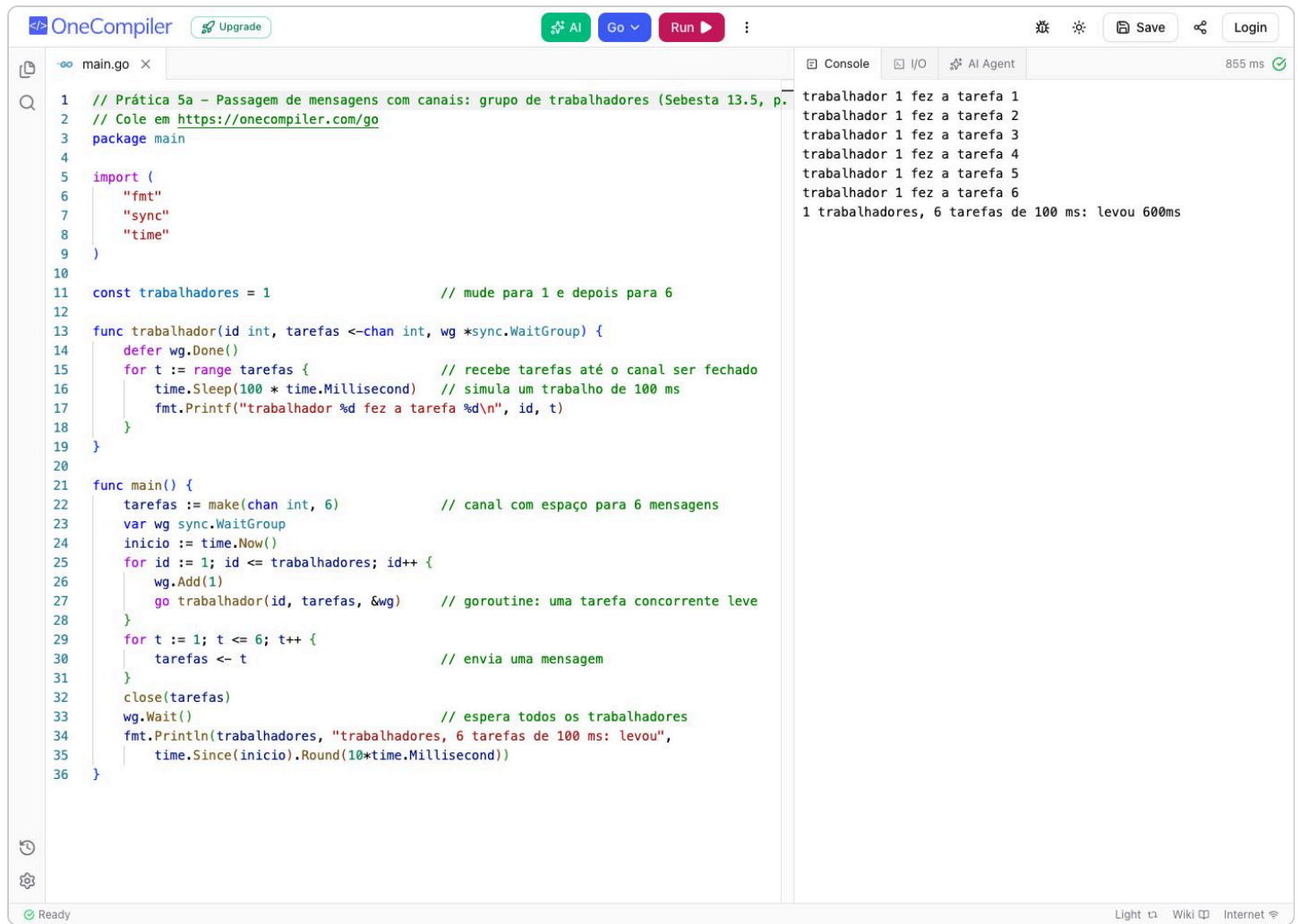
Tempo com 3 trabalhadores

200ms

Tempo com 6 trabalhadores

100ms

Saída do programa

não respondido

The screenshot shows the OneCompiler web IDE. The editor displays a Go program named `main.go`. The program defines a function `trabalhador` that takes an ID, a channel of tasks, and a wait group. It uses a `defer` statement to increment the wait group and a `for` loop to receive tasks from the channel, sleeping for 100ms between each task. The `main` function creates a channel with 6 tasks, launches 1 worker goroutine, and then waits for all workers to finish before printing the total time taken.

```
1 // Prática 5a - Passagem de mensagens com canais: grupo de trabalhadores (Sebesta 13.5, p.
2 // Cole em https://onecompiler.com/go
3 package main
4
5 import (
6     "fmt"
7     "sync"
8     "time"
9 )
10
11 const trabalhadores = 1 // mude para 1 e depois para 6
12
13 func trabalhador(id int, tarefas <-chan int, wg *sync.WaitGroup) {
14     defer wg.Done()
15     for t := range tarefas { // recebe tarefas até o canal ser fechado
16         time.Sleep(100 * time.Millisecond) // simula um trabalho de 100 ms
17         fmt.Printf("trabalhador %d fez a tarefa %d\n", id, t)
18     }
19 }
20
21 func main() {
22     tarefas := make(chan int, 6) // canal com espaço para 6 mensagens
23     var wg sync.WaitGroup
24     inicio := time.Now()
25     for id := 1; id <= trabalhadores; id++ {
26         wg.Add(1)
27         go trabalhador(id, tarefas, &wg) // goroutine: uma tarefa concorrente leve
28     }
29     for t := 1; t <= 6; t++ {
30         tarefas <- t // envia uma mensagem
31     }
32     close(tarefas)
33     wg.Wait() // espera todos os trabalhadores
34     fmt.Println(trabalhadores, "trabalhadores, 6 tarefas de 100 ms: levou",
35         time.Since(inicio).Round(10*time.Millisecond))
36 }
```

The console output shows the execution results:

```
trabalhador 1 fez a tarefa 1
trabalhador 1 fez a tarefa 2
trabalhador 1 fez a tarefa 3
trabalhador 1 fez a tarefa 4
trabalhador 1 fez a tarefa 5
trabalhador 1 fez a tarefa 6
1 trabalhadores, 6 tarefas de 100 ms: levou 600ms
```

The interface includes a top bar with 'OneCompiler', 'Upgrade', 'Go', 'Run', and 'Login' buttons. The bottom status bar shows 'Ready', 'Light', 'Wiki', and 'Internet' icons.

OneCompiler Upgrade

AI Go Run

Save Login

main.go

538 ms

```
1 // Prática 5a - Passagem de mensagens com canais: grupo de trabalhadores (Sebesta 13.5, p.
2 // Cole em https://onecompiler.com/go
3 package main
4
5 import (
6     "fmt"
7     "sync"
8     "time"
9 )
10
11 const trabalhadores = 3 // mude para 1 e depois para 6
12
13 func trabalhador(id int, tarefas <-chan int, wg *sync.WaitGroup) {
14     defer wg.Done()
15     for t := range tarefas { // recebe tarefas até o canal ser fechado
16         time.Sleep(100 * time.Millisecond) // simula um trabalho de 100 ms
17         fmt.Printf("trabalhador %d fez a tarefa %d\n", id, t)
18     }
19 }
20
21 func main() {
22     tarefas := make(chan int, 6) // canal com espaço para 6 mensagens
23     var wg sync.WaitGroup
24     inicio := time.Now()
25     for id := 1; id <= trabalhadores; id++ {
26         wg.Add(1)
27         go trabalhador(id, tarefas, &wg) // goroutine: uma tarefa concorrente leve
28     }
29     for t := 1; t <= 6; t++ {
30         tarefas <- t // envia uma mensagem
31     }
32     close(tarefas)
33     wg.Wait() // espera todos os trabalhadores
34     fmt.Println(trabalhadores, "trabalhadores, 6 tarefas de 100 ms: levou",
35         time.Since(inicio).Round(10*time.Millisecond))
36 }
```

Console I/O AI Agent

trabalhador 3 fez a tarefa 1
trabalhador 1 fez a tarefa 2
trabalhador 2 fez a tarefa 3
trabalhador 2 fez a tarefa 6
trabalhador 3 fez a tarefa 4
trabalhador 1 fez a tarefa 5
3 trabalhadores, 6 tarefas de 100 ms: levou 200ms

Ready Light Wiki Internet

The screenshot shows the OneCompiler IDE interface. The editor displays a Go program named `main.go` with the following code:

```
1 // Prática 5a - Passagem de mensagens com canais: grupo de trabalhadores (Sebesta 13.5, p.
2 // Cole em https://onecompiler.com/go
3 package main
4
5 import (
6     "fmt"
7     "sync"
8     "time"
9 )
10
11 const trabalhadores = 6 // mude para 1 e depois para 6
12
13 func trabalhador(id int, tarefas <-chan int, wg *sync.WaitGroup) {
14     defer wg.Done()
15     for t := range tarefas { // recebe tarefas até o canal ser fechado
16         time.Sleep(100 * time.Millisecond) // simula um trabalho de 100 ms
17         fmt.Printf("trabalhador %d fez a tarefa %d\n", id, t)
18     }
19 }
20
21 func main() {
22     tarefas := make(chan int, 6) // canal com espaço para 6 mensagens
23     var wg sync.WaitGroup
24     inicio := time.Now()
25     for id := 1; id <= trabalhadores; id++ {
26         wg.Add(1)
27         go trabalhador(id, tarefas, &wg) // goroutine: uma tarefa concorrente leve
28     }
29     for t := 1; t <= 6; t++ {
30         tarefas <- t // envia uma mensagem
31     }
32     close(tarefas)
33     wg.Wait() // espera todos os trabalhadores
34     fmt.Println(trabalhadores, "trabalhadores, 6 tarefas de 100 ms: levou",
35         time.Since(inicio).Round(10*time.Millisecond))
36 }
```

The console output on the right shows the execution results:

```
trabalhador 2 fez a tarefa 3
trabalhador 5 fez a tarefa 6
trabalhador 6 fez a tarefa 1
trabalhador 1 fez a tarefa 2
trabalhador 3 fez a tarefa 4
trabalhador 4 fez a tarefa 5
6 trabalhadores, 6 tarefas de 100 ms: levou 100ms
```

The status bar at the bottom indicates 'Ready' and shows icons for Light, Wiki, and Internet.

5.1 Explique os três tempos. Por que este programa não precisa de `synchronized` nem de trava?

Com 1 trabalhador ele faz as 6 tarefas uma depois da outra, então 600 ms. Com 3 cada um faz 2 ao mesmo tempo que os outros, 200 ms. Com 6 cada um faz uma e todas rodam juntas, 100 ms. Não precisa de `synchronized` porque as goroutines não mexem em nenhuma variável compartilhada, elas só recebem as tarefas pelo canal, e o canal já garante que cada tarefa vai pra um trabalhador só.

Saídas (antes e depois da troca)

não respondido

OneCompiler

Upgrade

AI

Go

Run

:

Save

Login

main.go

1 // Prática 5b - Um canal sem ninguém do outro lado
2 // Cole em <https://onecompiler.com/go>
3 package main
4
5 import "fmt"
6
7 func main() {
8 ch := make(chan int) // canal SEM espaço (sem buffer)
9 ch <- 1 // envia... e espera alguém receber
10 fmt.Println(<-ch)
11 }

Console

I/O

AI Agent

Exit 1

fatal error: all goroutines are asleep - deadlock!

goroutine 1 [chan send]:
main.main()
/box/main.go:9 +0x38
exit status 2

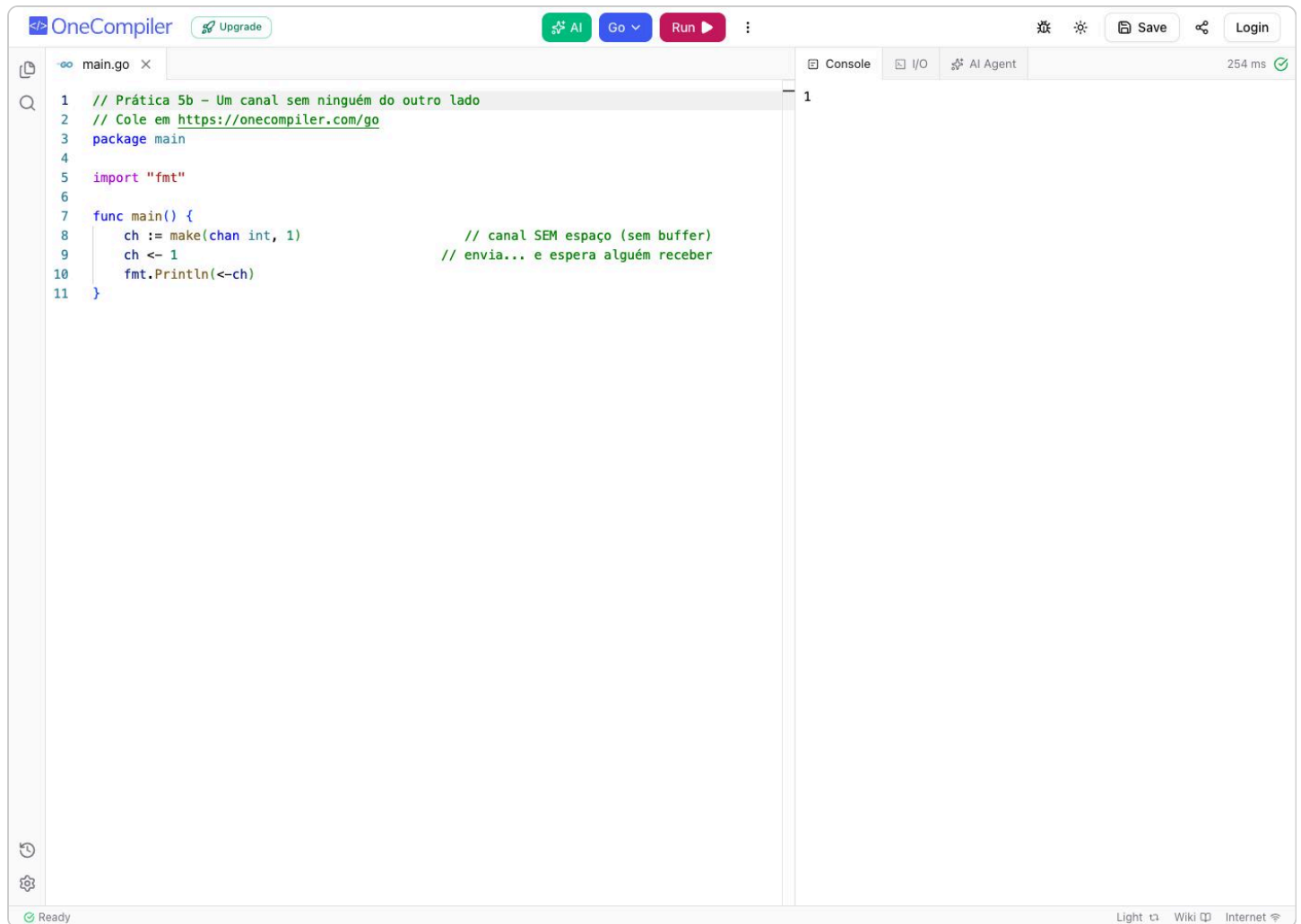
Debug this error

Ready

Light

Wiki

Internet



```
1 // Prática 5b - Um canal sem ninguém do outro lado
2 // Cole em https://onecompiler.com/go
3 package main
4
5 import "fmt"
6
7 func main() {
8     ch := make(chan int, 1)           // canal SEM espaço (sem buffer)
9     ch <- 1                          // envia... e espera alguém receber
10    fmt.Println(<-ch)
11 }
```

1

5.2 Por que o primeiro programa trava? Por que o canal com espaço para 1 mensagem resolve? (Dica: num canal sem espaço, quem envia espera alguém receber.)

No canal sem buffer quem envia fica esperando alguém receber. Só que a única goroutine é a main, e ela está parada no envio, então nunca chega na linha que recebe. O Go percebe que todo mundo está parado e dá o erro de deadlock. Com buffer de 1, o valor fica guardado no canal e a main continua, depois ela mesma lê e imprime 1.

Prática 6: O compilador como guarda

Previsão: Este programa faz uma thread somar 1 a uma variável de main. O que você acha que acontece?

Acho que imprime 1.

Saídas (o erro e o resultado com move)

não respondido

The image shows a screenshot of the OneCompiler web-based IDE. The editor displays a Rust program named `HelloWorld.rs` with the following code:

```
1 // Prática 6a - Rust: tentando compartilhar uma variável entre threads
2 // Cole em https://onecompiler.com/rust
3 use std::thread;
4
5 fn main() {
6     let mut contador = 0;
7     let t = thread::spawn(|| {
8         contador += 1; // a thread altera a variável de main
9     });
10    t.join().unwrap();
11    println!("{}", contador);
12 }
```

The right-hand pane shows the console output, which contains several Rust compiler error messages:

```
error[E0502]: cannot borrow `contador` as immutable because it is also borrowed as mutable
--> HelloWorld.rs:11:20
|
|         let t = thread::spawn(|| {
|         -                                     -- mutable borrow occurs here
|         _____|
|         |
|         |         contador += 1; // a thread altera a variável de main
|         |         ----- first borrow occurs due to use of `contador` in closure
|         |         });
|         |         _____ argument requires that `contador` is borrowed for `static`
10 |         t.join().unwrap();
11 |         println!("{}", contador);
|         ~~~~~~ immutable borrow occurs here
note: requirement that the value outlives `static` introduced here
-->
/rustc/48a229ceaeafd4985c50990b14116b6d856af0985/library/std/src/

error: aborting due to 2 previous errors

Some errors have detailed explanations: E0373, E0502.
For more information about an error, try `rustc --explain E0373`.
```

The screenshot shows the OneCompiler IDE interface. The editor on the left contains a Rust file named `HelloWorld.rs` with the following code:

```
1 // Prática 6a - Rust: tentando compartilhar uma variável entre threads
2 // Cole em https://onecompiler.com/rust
3 use std::thread;
4
5 fn main() {
6     let mut contador = 0;
7     let t = thread::spawn(move || {
8         contador += 1; // a thread altera a variável de main
9     });
10    t.join().unwrap();
11    println!("{}", contador);
12 }
```

The right-hand pane shows the console output, which includes two warnings from the Rust compiler:

```
warning: value captured by `contador` is never read
--> HelloWorld.rs:8:9
|
8 |         contador += 1; // a thread altera a variável de main
|         ~~~~~
|
= help: did you mean to capture by reference instead?
= note: `[warn(unused_assignments)]` (part of `[warn(unused)]`) on by default

warning: value assigned to `contador` is never read
--> HelloWorld.rs:8:9
8 |         contador += 1; // a thread altera a variável de main
|         ~~~~~
|
= help: maybe it is overwritten before being read?

warning: 2 warnings emitted

0
```

The bottom status bar indicates the IDE is 'Ready' and shows icons for Light mode, Wiki, and Internet connectivity.

Saída do programa

não respondido

```
1 // Prática 6b - Rust: compartilhar com Arc (contagem de referências) e Mutex (exclusão mútua)
2 // Cole em https://onecompiler.com/rust
3 use std::sync::{Arc, Mutex};
4 use std::thread;
5
6 fn main() {
7     let contador = Arc::new(Mutex::new(0)); // compartilhado e protegido
8     let mut threads = Vec::new();
9     for _ in 0..4 {
10         let c = Arc::clone(&contador);
11         threads.push(thread::spawn(move || {
12             for _ in 0..100_000 {
13                 *c.lock().unwrap() += 1; // trava, soma e destrava
14             }
15         }));
16     }
17     for t in threads { t.join().unwrap(); }
18     println!("contador = {}", *contador.lock().unwrap());
19 }
```

6.1 Com move, o programa compila mas imprime 0. Por quê? (Dica: move faz a thread levar uma cópia do número.)

Porque o contador é um inteiro, então o move faz uma cópia dele pra dentro da thread. A thread soma 1 na cópia dela e o contador da main continua 0. O próprio compilador dá um warning falando que o valor nunca é lido. Pra funcionar de verdade tem que usar Arc com Mutex, como na 6b, que deu 400000.

6.2 Compare com a Prática 1: em que momento o erro foi descoberto em Java e em Rust? Qual é a vantagem e qual é o custo para quem programa?

No Java o erro só apareceu rodando, e com n pequeno nem apareceu. No Rust o programa nem compilou. A vantagem é que esse tipo de erro não chega em produção. O custo é que é mais difícil de programar, tem que entender ownership, Arc, Mutex e ficar brigando com o compilador até ele aceitar.

Prática 7: Uma thread só: o laço de eventos

Previsão: Em que ordem você acha que as letras A, B, C e D serão impressas?

A D B C

Saída do programa

não respondido

```
1 // Prática 7 - JavaScript: uma única thread e um laço de eventos
2 // Cole em https://onecompiler.com/nodejs
3 console.log("A");
4 setTimeout(() => console.log("B"), 0);
5 Promise.resolve().then(() => console.log("C"));
6 console.log("D");
7
8 function ocupado(ms) { // laço que NÃO devolve o controlador
9   const fim = Date.now() + ms;
10  while (Date.now() < fim) { }
11 }
12
13 setTimeout(() => {
14   const t0 = Date.now();
15   setTimeout(() => console.log("E: timer de 10 ms rodou depois de " + (Date.now() - t0)), 10);
16   ocupado(300); // trava a única thread por 300 ms
17 }, 0);
```

Console

```
A
D
C
B
E: timer de 10 ms rodou depois de 300 ms
```

7.1 Em que ordem as letras saíram? Por que B, com tempo 0, saiu depois de D e de C?

Saiu A, D, C, B. A e D saem primeiro porque são código normal. O B e o C só rodam depois que o código principal acaba. O C vem antes do B porque promessa vai pra fila de microtarefas, que roda logo em seguida, e o setTimeout vai pra fila de timers, que só roda na próxima volta do laço de eventos. Então tempo 0 não quer dizer que roda na hora.

7.2 O timer de 10 ms (linha E) rodou bem depois de 10 ms. Por quê? O que isso significa para um servidor Node.js que atende muitos usuários e faz um cálculo pesado?

Porque a função ocupado ficou 300 ms segurando a única thread. O timer de 10 ms já tinha vencido, mas não tinha como rodar até a thread ficar livre. Num servidor Node isso seria ruim, porque enquanto um cálculo pesado está rodando nenhum outro usuário é atendido, o servidor inteiro trava. Teria que jogar esse cálculo pra uma worker thread ou outro processo.

Fechamento: Quem impede a corrida?

Java sem sincronização

Ninguém: há corrida

Java com synchronized

O programador, com sincronização

Go com canais

O projeto: não há memória compartilhada

Rust com Arc e Mutex

O compilador

JavaScript

Só existe uma thread

8.1 Um site de venda de ingressos tem 1 lugar restante, e duas pessoas clicam em Comprar ao mesmo tempo. Qual problema desta atividade pode acontecer? Como você o evitaria?

Condição de corrida. As duas compras podem ver que tem 1 lugar livre ao mesmo tempo e as duas confirmarem, vendendo o mesmo lugar duas vezes, igual o problema do contador++. Pra evitar, a verificação e a reserva tem que acontecer juntas, de forma atômica, por exemplo no banco com um UPDATE ... WHERE lugar ainda livre, ou uma transação com lock. Quem chegar depois recebe que está esgotado.

8.2 Em uma frase: o que mais surpreendeu você nesta atividade?

Ver o contador do Java dar certo com 1000 e errar feio com 1 milhão, sendo literal o mesmo código.